

CarRental — Front-End Build

Week 3 work report: visual front end, mock data layer, and repository setup

Vincent Gallagher · September 28, 2026 · github.com/vegallag-compsci/car-rental-joe-wac

Summary

Built the complete visual front end for the CarRental site in React and Vite. All six planned pages render and are navigable. This is the presentation layer only — there is no authentication, no real booking logic, and no backend calls. Every page draws from a single hard-coded mock data file that uses our real database column names, so it can be swapped for Supabase queries without rewriting components.

METRIC	RESULT
Files created	16 new source files
Files modified	4 (App, main, index.css, index.html)
Total committed	32 files, 3,851 insertions
Routes working	6 planned + 1 catch-all 404
Build status	Passing — no errors, no lint warnings

Pages built

ROUTE	PAGE AND WHAT IT DOES
/	Hero, search bar (dates + category), 5 category tiles with live 'from' pricing, 3-step how-it-works strip
/cars	Filter bar (dates, category, sort by price) over a responsive car grid; filtering and sorting both work
/cars/:id	Large image, 8-field spec list, booking box with date pickers computing days x daily rate = total
/bookings	Three mock bookings with dates, totals, and colored status badges; Cancel flips pending/confirmed
/login	Centered card — email, password, Log in, Continue with Google. Visual only.
/admin	Fleet table with category, rate, and a working active toggle; Add car and Edit buttons
*	404 fallback, added so bad URLs don't render a blank page

Deliverables

Mock data layer

`src/data/mockCars.js` — the single file to replace when Supabase is connected.

- 12 cars spread across all 5 categories, rates from \$35 to \$250 per day
- 5 categories (Economy, Sedan, SUV, Luxury, Van) and 3 sample bookings

- Field names match our planned schema exactly: `category_id`, `daily_rate`, `is_active`, `pickup_at`, `return_at`, `total_price`
- Helper functions (`getCarById`, `daysBetween`, `formatMoney`) so pages never handle raw data shapes

Components — 7 reusable pieces

`CarCard` `CategoryBadge` `StatusBadge` `SearchBar` `Nav` `Footer` `ScrollToTop`

Each is deliberately small and single-purpose so every team member can read and edit them without tracing through the whole app.

Styling and design system

- Plain CSS only — no Tailwind, no UI libraries, so the whole team can edit it directly
- Black and white palette with soft grays; frosted 'liquid glass' panels using `backdrop-filter`, rounded corners, subtle hover lift on cards and buttons
- Light *and* dark mode via `prefers-color-scheme` — all colors are CSS custom properties defined once in `:root` and overridden for dark
- Fully responsive: single column on phones, multi-column grid on desktop
- Sticky frosted nav that collapses to a menu button under 760px
- Honors `prefers-reduced-motion` for accessibility

Architecture decision: filters live in the URL

Browse-page filters are stored as query parameters (`/cars?category=3&sort=price-asc`) rather than in component state. Filtered views are therefore shareable and bookmarkable, and the params map directly onto future Supabase `.eq()` and `.order()` calls.

Verification

A successful build only proves the code compiles, not that pages render. Every route was server-rendered to catch runtime errors, and the output was checked against expected values:

CHECK	EXPECTED	RESULT
Production build	No errors	Pass
Linters (oxlint)	No warnings	Pass
All 8 routes render	No runtime errors	Pass
Default car list	11 of 12 (1 inactive hidden)	11 shown
SUV category filter	Only the 3 SUVs	\$89, \$76, \$95
Sort: price ascending	\$35 up to \$250	Correct order
Sort: price descending	\$250 down to \$35	Correct order
Price math (BMW, 3 days)	$\$165 \times 3 = \495	\$495

Note: the inactive car (Kia Rio) is correctly hidden from customers on `/cars` while still appearing in the admin table — confirming the `is_active` flag works as intended in both directions.

Version control

- Repository initialized and pushed to `github.com/vegallag-compsci/car-rental-joe-wac` on branch `main`
- `.gitignore` added — excludes `node_modules`, build output, and `.env` files so nobody commits Supabase keys later
- README written covering setup commands, the route table, project layout, the database columns, and backend integration notes
- Verified the push landed: remote and local both at commit `647f9df`, working tree clean

Team note: `react-router-dom` was missing from `package.json` and had to be installed. Anyone pulling the repo must run `npm install` inside `my-react-app/` or the app will not start.

Known limitations before the backend

These are deliberate — they are the seams where Supabase gets connected.

ITEM	CURRENT STATE AND WHAT'S NEEDED
Date filtering	Dates are collected but filter nothing. Real availability needs a query for overlapping booking ranges — the largest outstanding piece of logic.
Reserve button	Navigates to <code>/bookings</code> . Needs to write a real booking record.
Bookings & Admin	Hold local copies of mock data so buttons visibly respond; both reset on refresh. Replace with fetched data plus mutations.
<code>is_active</code> guard	Respected on <code>/cars</code> , but hitting <code>/cars/:id</code> directly still renders an inactive car. Needs a guard.
Money format	Whole dollars. If the schema uses cents, <code>formatMoney</code> is the single place to change.
SPA routing	<code>BrowserRouter</code> needs a host-side catch-all rewrite to <code>index.html</code> , or direct links like <code>/cars/9</code> will 404 in production.
Images	All placeholders from <code>placeholder.co</code> . Real photography will change how the cards look.

Next steps

- Stand up the Supabase project; create `cars`, `car_categories`, and `bookings` using the column names already in the mock data
- Replace `mockCars.js` imports with Supabase queries, pushing existing URL params into the query instead of filtering client-side
- Implement auth — email/password plus Google OAuth — against the existing login UI
- Build availability logic so the date pickers actually filter results
- Make Reserve write a real booking; replace local state in Bookings and Admin with real mutations
- Add the `is_active` guard and configure the SPA rewrite before deploying